Guía semana 4 - Autenticación

DJANGO · GUÍA DE PRÁCTICA

Paso 1 · Crear un usuario desde el panel admin

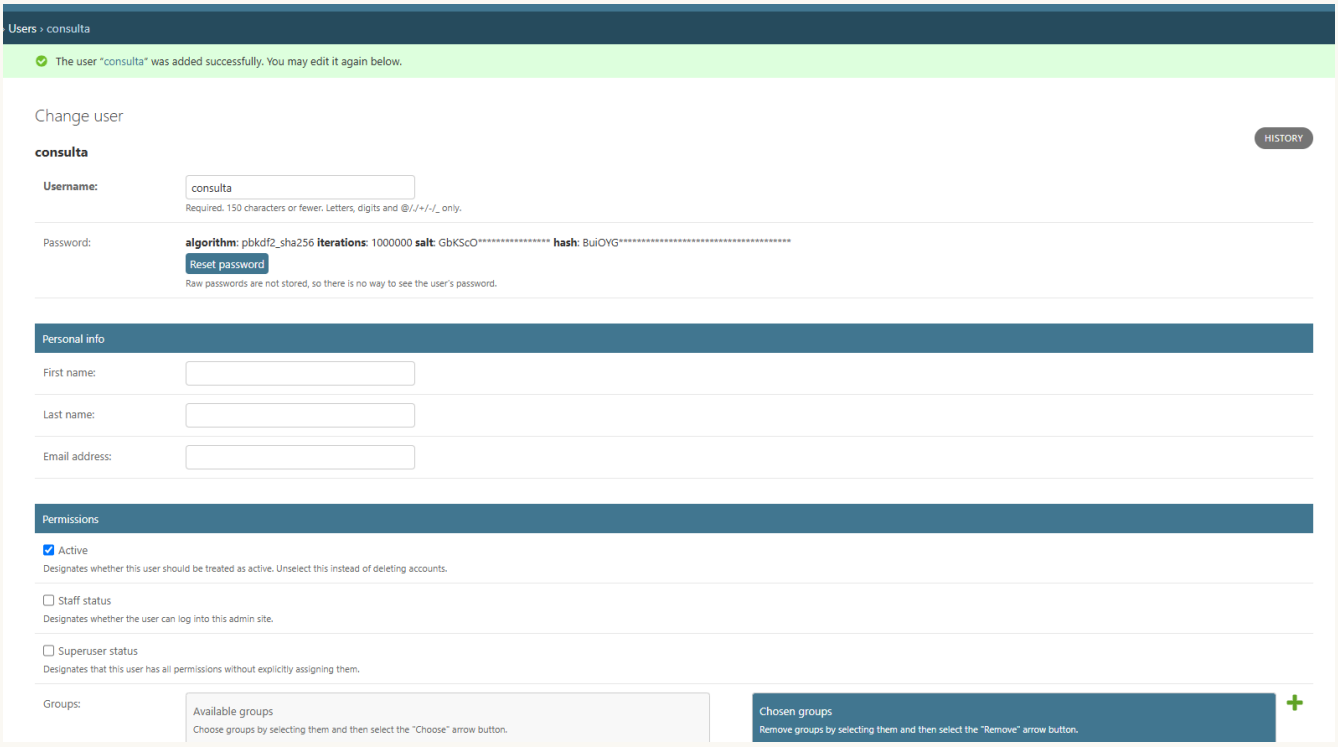
Ingresamos al panel de administración de Django y abrimos **Authentication and Authorization > Users**. Luego seleccionamos **Add user** para crear un usuario nuevo.

```
http://localhost:8000/admin/
```

En el formulario escribimos el nombre de usuario **consulta**, asignamos una contraseña y la confirmamos. Después presionamos **Save**.

Django crea el usuario y muestra un aviso verde de confirmación. La contraseña no se guarda como texto visible: el panel muestra únicamente información de su hash.

La opción **Active** permite que la cuenta pueda utilizarse. Para un usuario normal no es necesario marcar **Staff status** ni **Superuser status**.



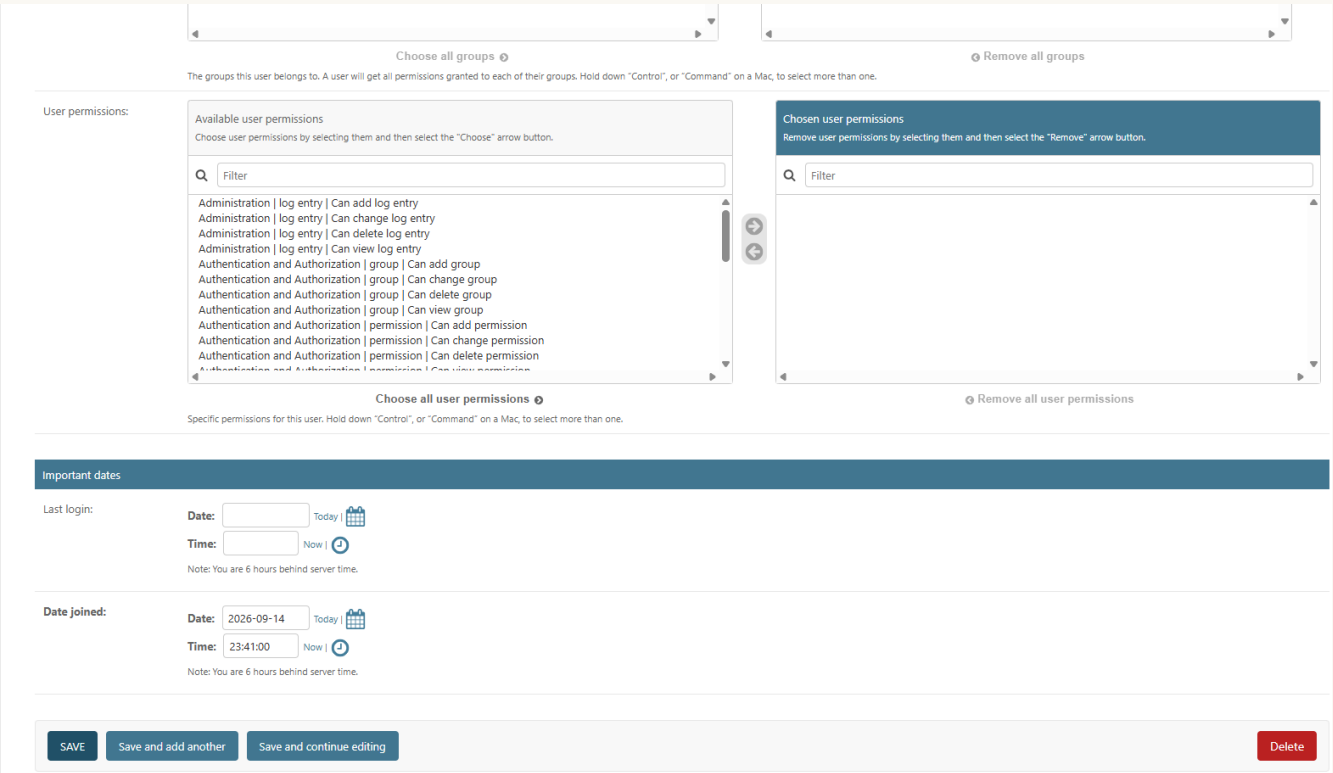
Revisar permisos y guardar

Paso 1 · Continuación de la creación

En la parte inferior del formulario aparecen los grupos y permisos disponibles. Podemos dejarlos vacíos si la cuenta solamente se utilizará para hacer consultas desde la aplicación.

Si el usuario necesita permisos concretos, seleccionamos cada permiso en la columna **Available user permissions** y lo trasladamos a **Chosen user permissions**.

Finalmente, presionamos **Save** para guardar la configuración. El botón **Save and continue editing** también guarda los cambios, pero mantiene abierto el formulario.



Habilitar la autenticación por token

Paso 2 · Registrar authtoken en settings.py

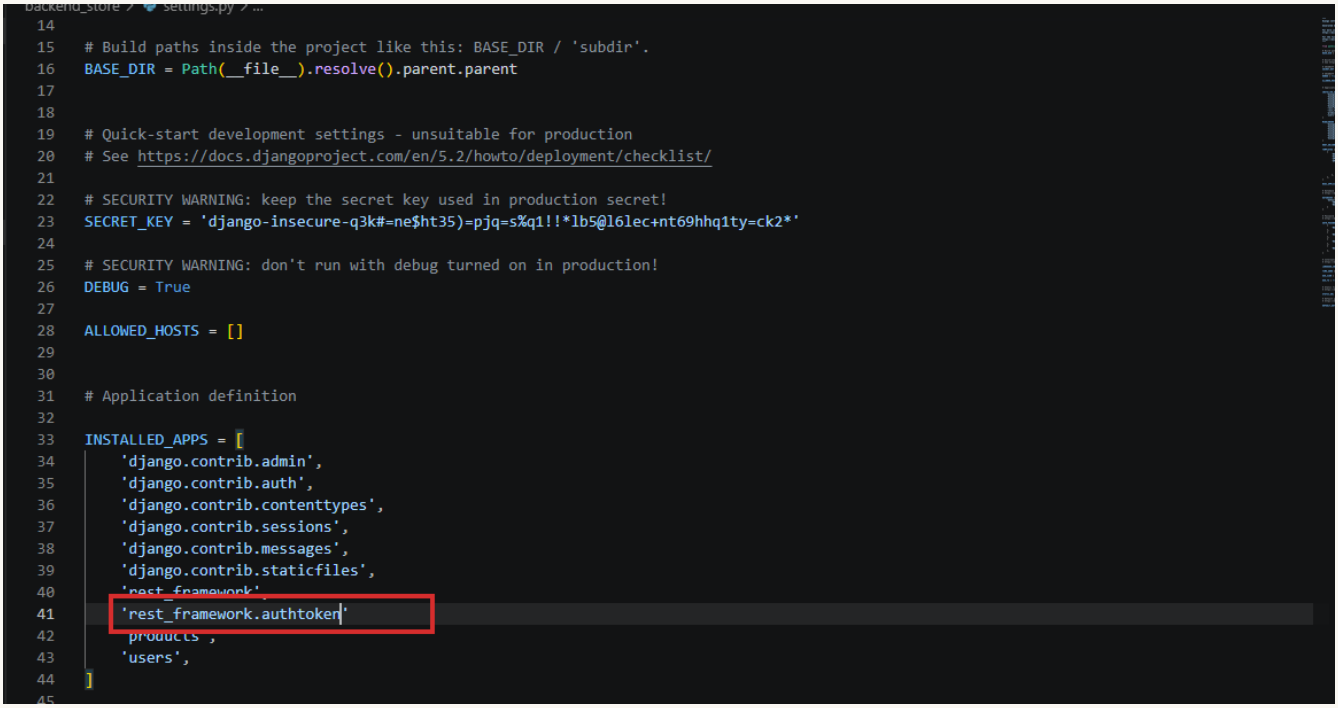
Nos dirigimos a **backend_store/settings.py** y agregamos **'rest_framework.authtoken'** dentro de **INSTALLED_APPS**.

```
INSTALLED_APPS = [
# Aplicaciones de Django...
'rest_framework',
'rest_framework.authtoken',
'products',
'users',
]
```

Esta aplicación incorpora el modelo de tokens de Django REST framework. Cada token puede asociarse con un usuario y utilizarse posteriormente para autenticar solicitudes a la API.

Lo colocamos justo debajo de **'rest_framework'** para mantener un mejor orden visual. Su posición dentro de **INSTALLED_APPS** no afecta el funcionamiento en este caso.

Después de guardar **settings.py**, la aplicación queda registrada en el proyecto y podremos preparar sus tablas mediante las migraciones.



Aplicar las migraciones

Paso 3 · Crear las tablas de authtoken

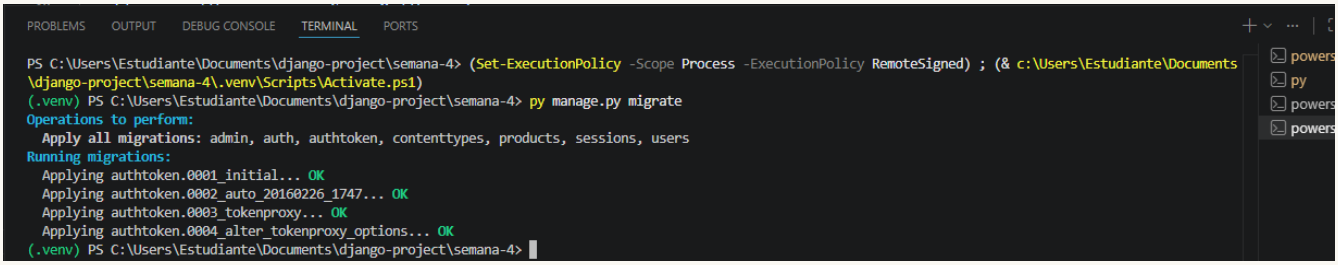
Seguidamente, abrimos la terminal con el entorno virtual activo y ejecutamos la migración para aplicar los cambios pendientes en la base de datos.

```
py manage.py migrate
```

Django revisa las aplicaciones registradas en **INSTALLED_APPS** y encuentra las migraciones de **authtoken**. Después crea o actualiza las tablas que necesita esta aplicación.

En la terminal aparecen migraciones como **authtoken.0001_initial** y las siguientes actualizaciones. El mensaje **OK** confirma que cada una se aplicó correctamente.

Al finalizar, volvemos a ver el indicador **(.venv)** y el cursor de PowerShell. Esto indica que el comando terminó sin errores.



Crear la ruta para obtener tokens

Paso 4 · Configurar backend_store/urls.py

Nos dirigimos a **backend_store/urls.py** e importamos la vista que proporciona Django REST framework para obtener los tokens de autenticación.

```
from rest_framework.authtoken.views import obtain_auth_token

urlpatterns = [
# Otras rutas...
path('api/token/', obtain_auth_token, name='api_token'),
]
```

1. Importación: obtain_auth_token es una vista preparada por DRF. Se encarga de recibir un nombre de usuario y una contraseña, validar las credenciales y devolver el token correspondiente.

2. Ruta: path('api/token/', ...) crea el endpoint al que el cliente enviará la solicitud. La dirección completa será **http://localhost:8000/api/token/**.

3. Nombre: name='api_token' permite identificar esta ruta por su nombre dentro de Django.

En resumen, conectamos una ruta con la vista encargada de generar u obtener el token del usuario. Para utilizarla se envían las credenciales mediante una solicitud **POST**.



Probar la generación del token

Paso 5 · Enviar las credenciales desde Postman

Seguidamente, abrimos **Postman**, seleccionamos el método **POST** y escribimos la URL del endpoint creado en el paso anterior.

```
POST http://localhost:8000/api/token/
```

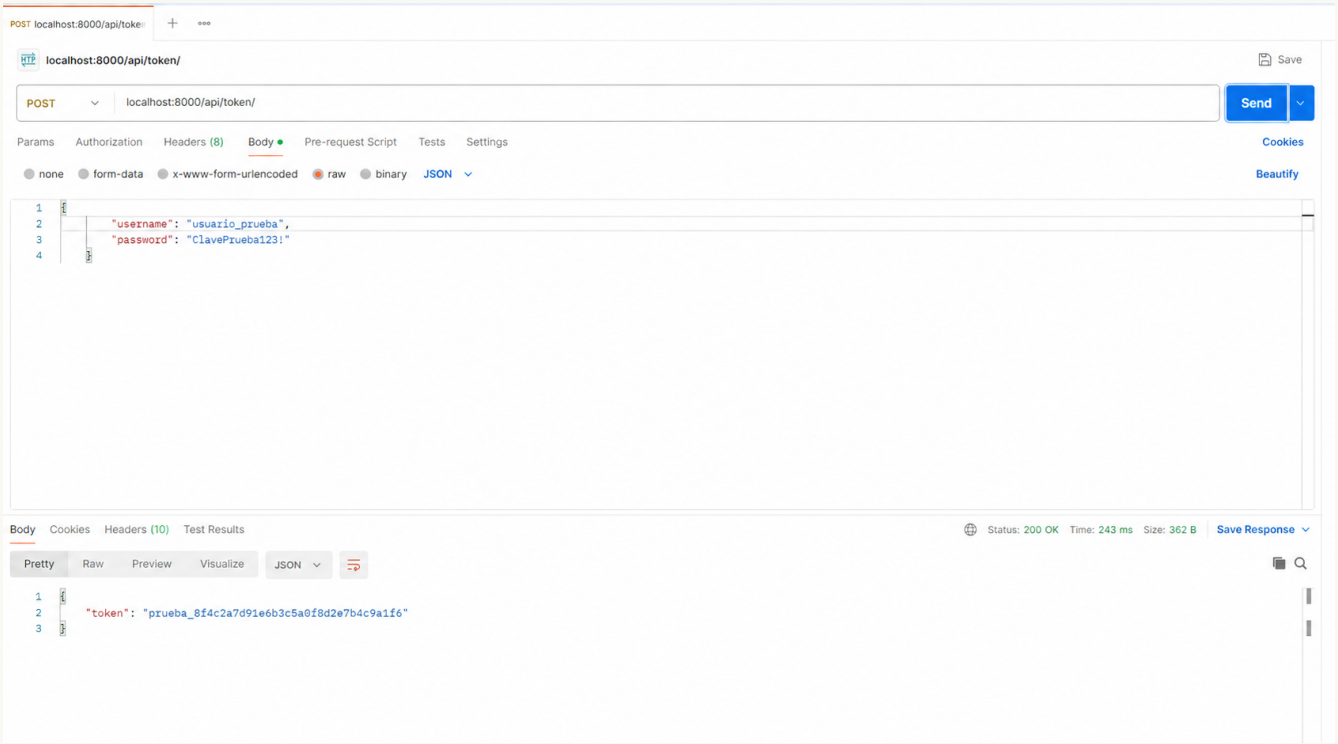
Utilizamos POST porque enviaremos las credenciales al servidor para solicitar el token. En **Body** seleccionamos **raw**, elegimos **JSON** y enviamos un objeto con el nombre de usuario y la contraseña.

```
{
"username": "usuario_prueba",
"password": "ClavePrueba123!"
}
```

Al presionar **Send**, la vista valida las credenciales. Si son correctas, Postman muestra el estado **200 OK** y la respuesta contiene una propiedad **token**. Esto confirma que el token se obtuvo correctamente.

Seguridad: el token funciona como una credencial para autenticarse en la API. Debemos evitar compartirlo, publicarlo o guardarlo en archivos públicos. Si otra persona obtiene el token, podría realizar solicitudes con los permisos del usuario asociado.

En la captura se sustituyeron el usuario, la contraseña y el token originales por valores ficticios de prueba. Así podemos observar el formato completo sin exponer credenciales reales.



Preparar un endpoint protegido

Paso 6 · Importar autenticación y permisos

Ahora crearemos un endpoint protegido para consultar el perfil del usuario autenticado. Nos dirigimos a **products/views.py** y agregamos los imports necesarios.

```
from rest_framework.authentication import TokenAuthentication
from rest_framework.decorators import (
api_view,
authentication_classes,
permission_classes,
)
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
```

TokenAuthentication busca el token enviado en la solicitud y, si es válido, identifica al usuario relacionado con él.

authentication_classes permite indicar qué sistema de autenticación usará la vista. En este caso utilizaremos autenticación por token.

permission_classes define las reglas de acceso. **IsAuthenticated** comprueba que la solicitud pertenezca a un usuario autenticado.

api_view convierte la función en una vista de DRF y limita los métodos HTTP permitidos. Esto evita escribir manualmente gran parte de la lógica repetitiva necesaria para procesar solicitudes y respuestas HTTP.

Los autenticadores y permisos se ejecutan automáticamente antes del contenido del endpoint; no necesitamos llamarlos dentro de la función.

Crear el endpoint de perfil

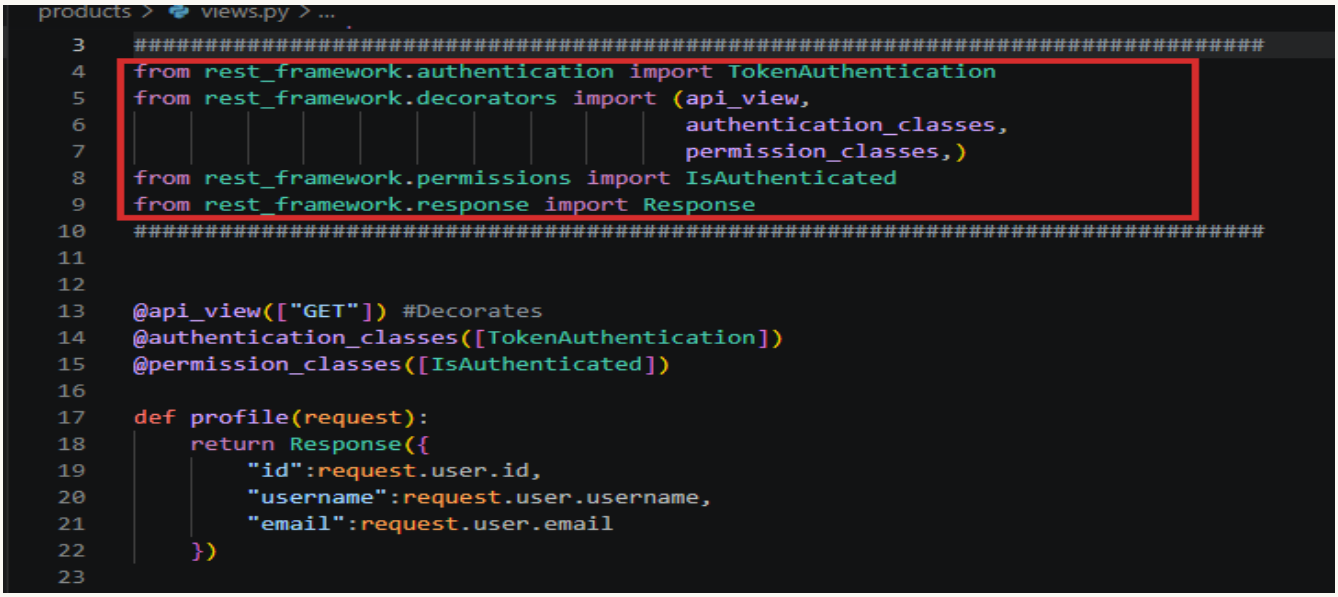
Paso 6 · Aplicar los decoradores

```
@api_view(['GET'])
@authentication_classes([TokenAuthentication])
@permission_classes([IsAuthenticated])
def profile(request):
return Response({
'id': request.user.id,
'username': request.user.username,
'email': request.user.email,
})
```

La diferencia con los endpoints anteriores está en los dos decoradores de seguridad. **@authentication_classes([TokenAuthentication])** comprueba que la solicitud incluya un token válido.

Después, **@permission_classes([IsAuthenticated])** responde a la pregunta «¿el usuario está autenticado?». Si no lo está, DRF impide que la función continúe.

Cuando el token es válido, DRF coloca automáticamente el usuario autenticado en **request.user**. Por eso podemos obtener su ID, nombre de usuario y correo sin buscarlo manualmente.



Registrar la ruta del perfil

Paso 6 · Continuación en products/urls.py

Seguidamente, nos dirigimos a **products/urls.py** y agregamos el path que conecta la dirección **api/profile/** con la vista protegida **profile**.

```
path('api/profile/', views.profile, name='details_profile'),
```

Cuando Django recibe una solicitud en esta ruta, llama a **views.profile**. Antes de ejecutar la función, DRF aplica automáticamente la autenticación por token y el permiso **IsAuthenticated** configurados en la vista.

Como las URLs de products están incluidas bajo **products/** en el archivo principal, la dirección completa del endpoint será **http://localhost:8000/products/api/profile/**.

El nombre **details_profile** permite identificar y reutilizar esta ruta desde Django sin escribir la dirección completa.



Probar el perfil sin token

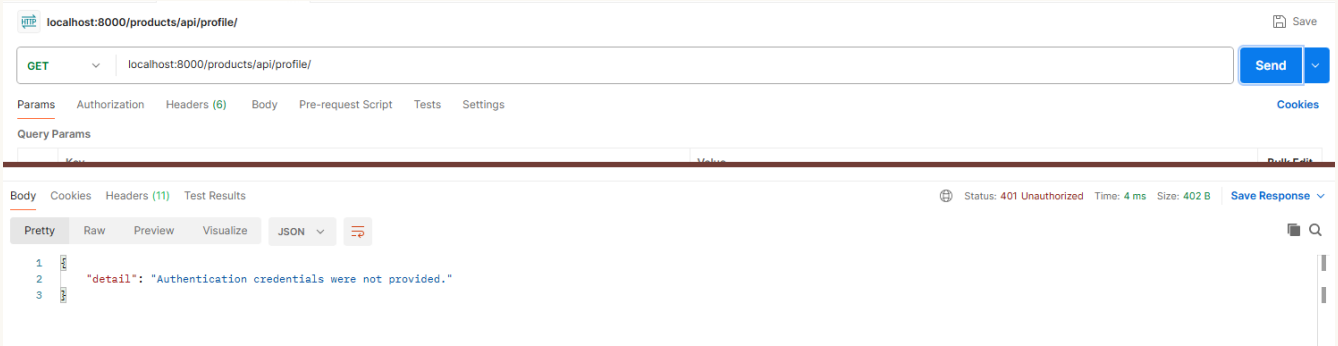
Paso 6 · Comprobar la protección del endpoint

Vamos a Postman y realizamos una primera solicitud **GET** sin enviar ningún token. Esta prueba sirve para comprobar que la URL funciona y que el endpoint realmente está protegido.

```
GET http://localhost:8000/products/api/profile/
```

La respuesta muestra el estado **401 Unauthorized** y el mensaje **Authentication credentials were not provided.** Nosotros no escribimos ese mensaje: Django REST framework lo genera automáticamente al aplicar los decoradores de autenticación y permisos.

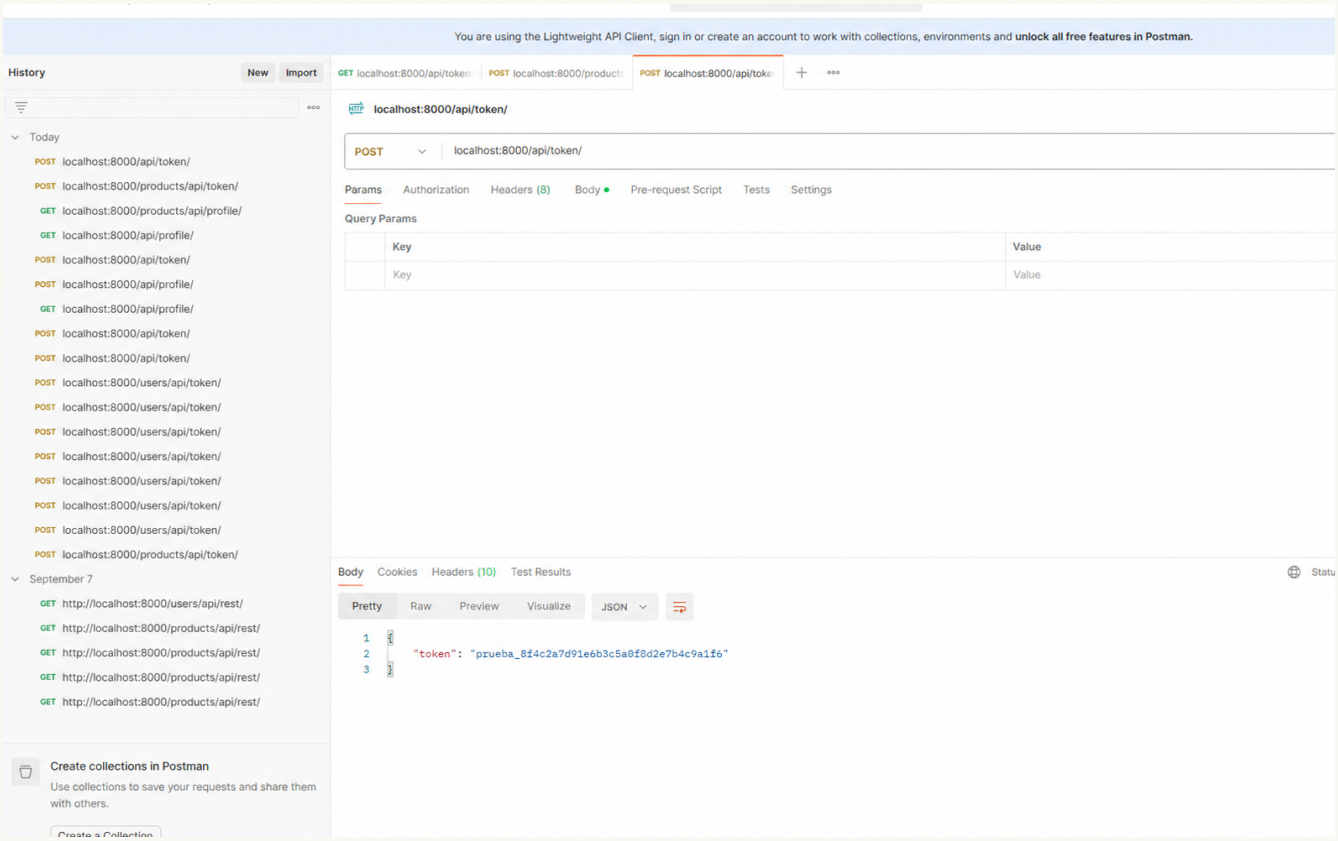
Esto confirma dos cosas: Django encontró la ruta y **IsAuthenticated** impidió el acceso porque la solicitud no contenía credenciales válidas.



Enviar el token desde Postman

Paso 6 · Autenticar la solicitud GET

Seguidamente, generamos el token con una solicitud POST a **/api/token/** y copiamos el valor devuelto. En la captura se utiliza un token ficticio de prueba.



Volvemos a la solicitud del perfil, abrimos la pestaña **Headers** y activamos la opción para mostrar los headers ocultos. Postman muestra los encabezados que acompañan la solicitud; nosotros agregaremos el header de autenticación.

```
Authorization: Token TU_TOKEN
```

La palabra **Token** debe escribirse antes del valor, separada por un espacio. Al presionar **Send**, DRF leerá este header, identificará al usuario y permitirá el acceso al perfil si el token es válido.

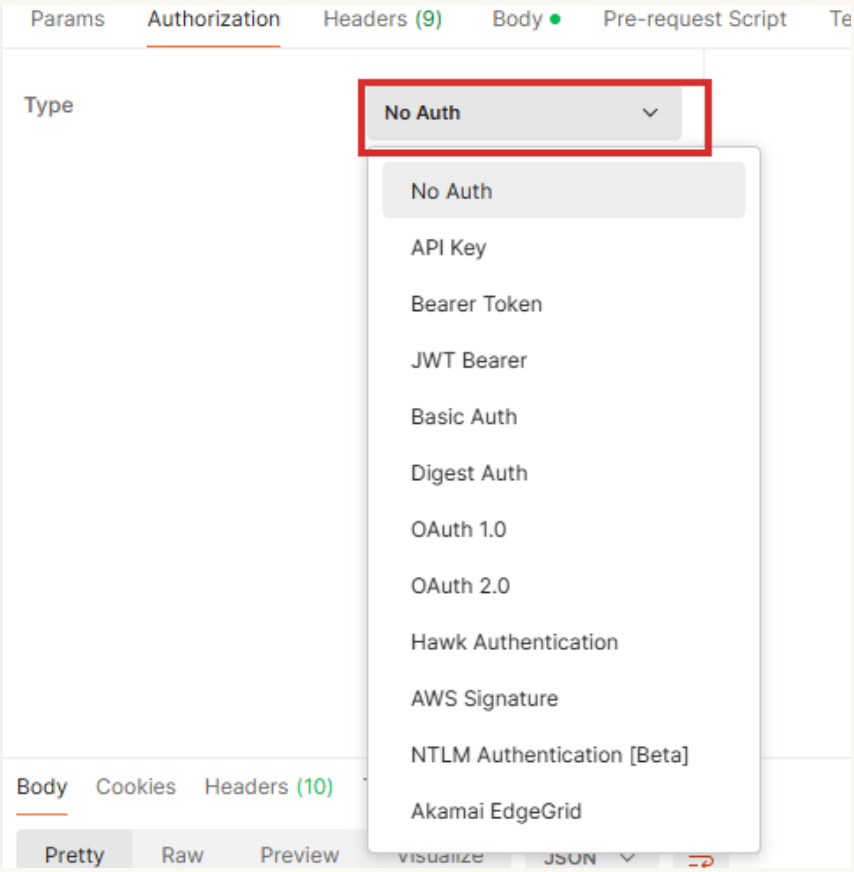
Recordar: utilizamos GET porque estamos consultando la información del perfil. El token viaja en los headers y no en el Body.

Seleccionar No Auth

Paso 6 · Preparar la autenticación manual

En la pestaña **Authorization** de Postman procuramos tener seleccionada la opción **No Auth**. Así evitamos que Postman aplique automáticamente otro método de autenticación, porque enviaremos el token manualmente desde Headers.

Esta configuración se utiliza para la solicitud GET del perfil protegido. Después de comprobarla, abrimos la pestaña **Headers** para agregar la credencial.



Agregar el token en Headers

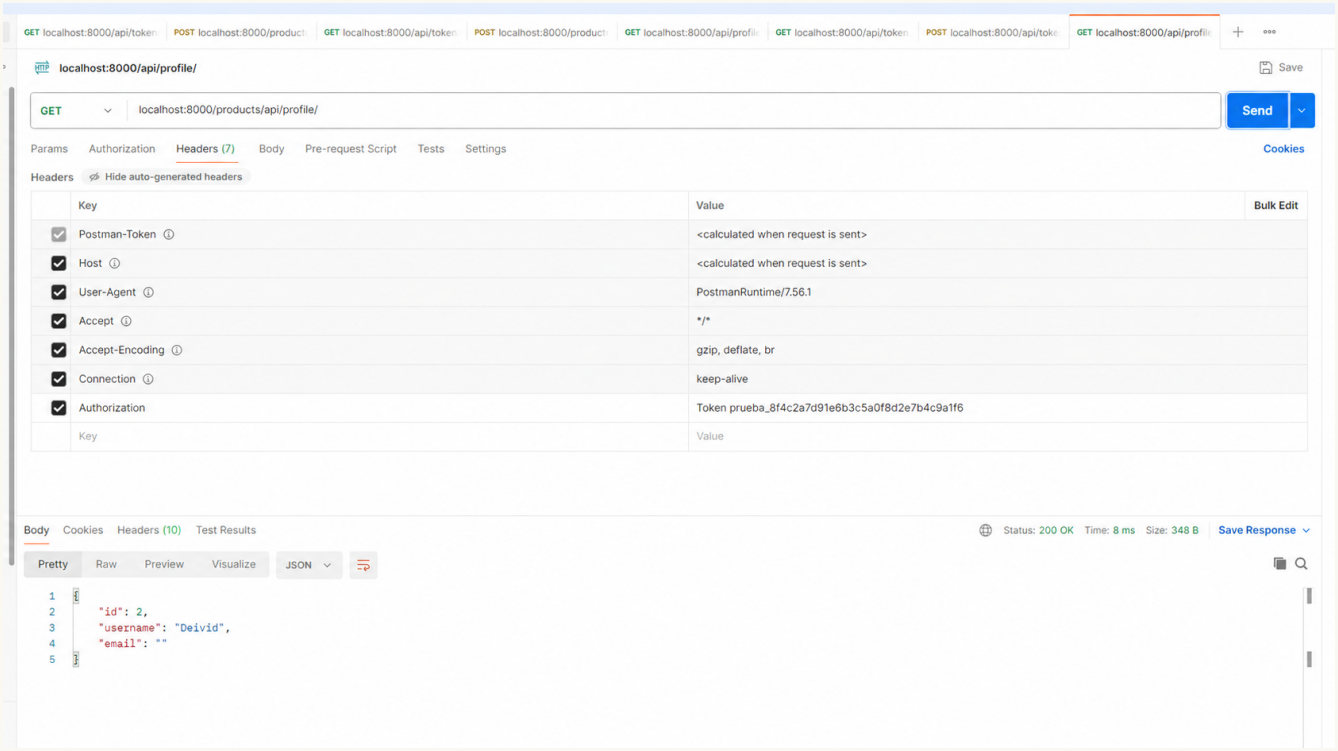
Paso 6 · Enviar una solicitud autenticada

En **Headers** agregamos una fila con la clave **Authorization**. Como valor escribimos la palabra **Token**, dejamos un espacio y pegamos los caracteres del token obtenido anteriormente.

```
Authorization: Token TU_TOKEN
```

Enviamos un **GET** a **/products/api/profile/**. Si el token pertenece a un usuario registrado y sigue siendo válido, DRF autoriza la solicitud y devuelve los datos del perfil, como el ID, el nombre de usuario y el correo.

La captura muestra una respuesta correcta con estado **200 OK**. El valor real del token se sustituyó por un ejemplo que comienza con **prueba_**.



Diferencia entre las dos URLs

Paso 6 · Obtener y validar el token

Aunque ambas rutas participan en la autenticación, cada una tiene una función y un método HTTP diferente.

Acción	Método y URL	Datos enviados
Obtener el token	**POST** http://localhost:8000/api/token/	Usuario y contraseña en el Body
Consultar el perfil protegido	**GET** http://localhost:8000/products/api/profile/	Authorization: Token TU_TOKEN en Headers

Primero enviamos las credenciales mediante POST a **/api/token/**. El servidor las verifica y responde con el token del usuario.

Luego copiamos ese valor y realizamos un GET a **/products/api/profile/** con el header **Authorization**. El endpoint valida el token, identifica al usuario y devuelve su perfil.

Conceptos clave del acceso protegido

Apoyo · Autenticación, autorización y respuestas HTTP

La solicitud protegida pasa por dos comprobaciones relacionadas, pero diferentes. Primero se identifica al usuario y después se decide si puede realizar la acción.

Autenticación	Autorización
Responde **¿quién es?** TokenAuthentication valida la credencial y asocia el usuario con **request.user**.	Responde **¿qué puede hacer?** Las clases de permisos, como IsAuthenticated, permiten o rechazan la operación.

Cómo interpretar 401 y 403

401 · Unauthorized	403 · Forbidden
La solicitud no pudo demostrar correctamente quién es el usuario. Puede faltar el token o ser inválido.	El usuario puede estar identificado, pero no tiene permiso para ejecutar esa operación.

En nuestra primera prueba del perfil recibimos **401** porque no enviamos el token. Un **403** aparecería cuando el usuario ya fue reconocido, pero una regla de permisos le impide realizar la acción solicitada.

Hacer que el sistema recuerde el inicio de sesión

Paso 7 · Crear un endpoint exclusivo para administradores

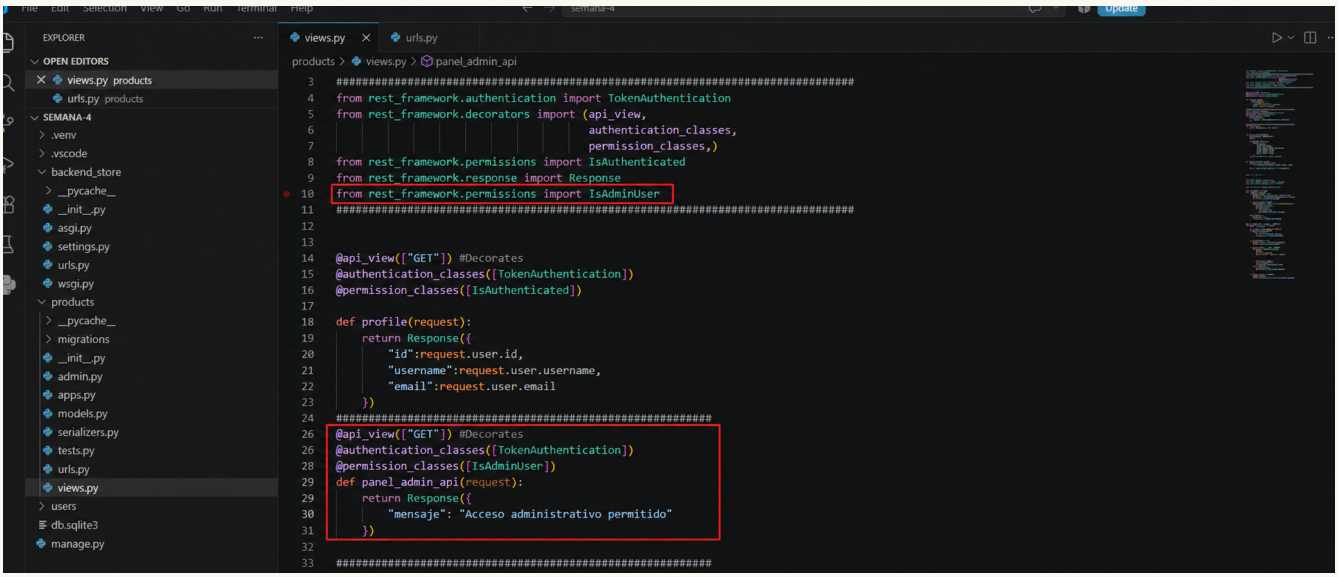
El cliente reutiliza el token en solicitudes posteriores para que el servidor pueda identificar al usuario. En este paso también comprobaremos si esa identidad pertenece a un usuario con acceso administrativo.

En **products/views.py** importamos **IsAdminUser**, una clase de permiso de Django REST framework que permite el acceso únicamente cuando **request.user.is_staff** es verdadero.

```
from rest_framework.permissions import IsAdminUser
```

Utilizamos los mismos decoradores del endpoint anterior. El cambio principal está en **@permission_classes([IsAdminUser])**, que reemplaza a IsAuthenticated para exigir que el usuario sea miembro del personal administrativo.

```
@api_view(['GET'])
@authentication_classes([TokenAuthentication])
@permission_classes([IsAdminUser])
def panel_admin_api(request):
return Response({
'mensaje': 'Acceso administrativo permitido'
})
```



Registrar la URL administrativa

Paso 7 · Agregar la ruta en products/urls.py

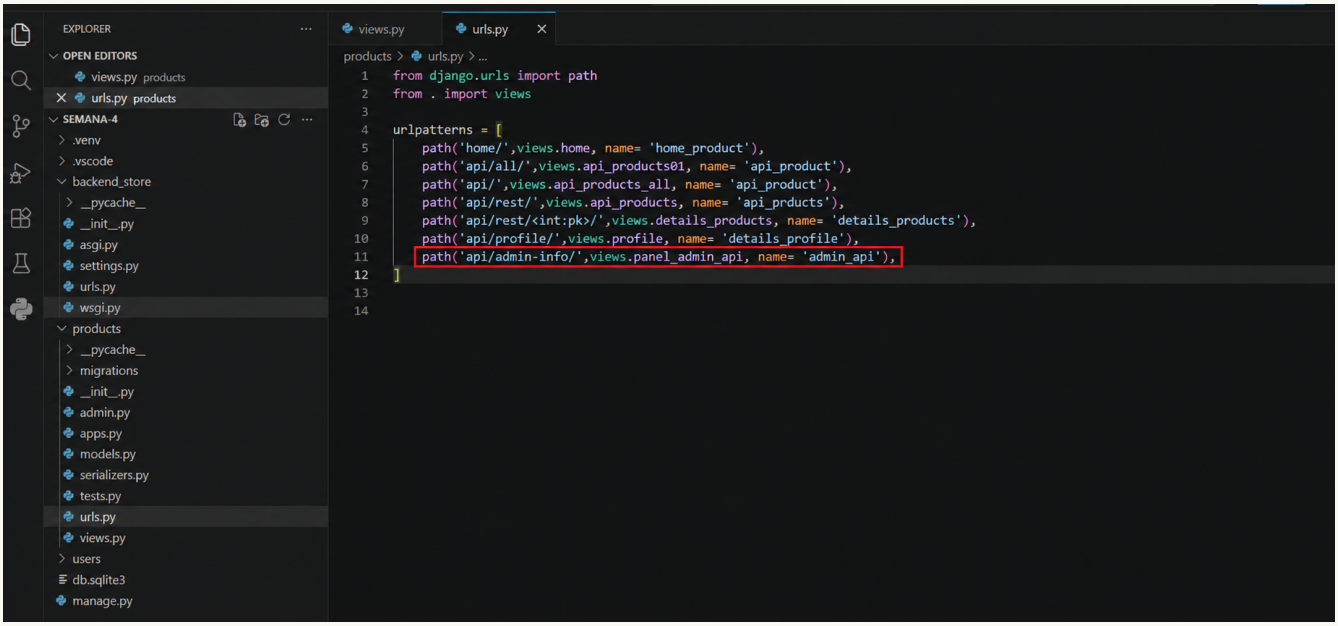
Después agregamos una ruta que conecte la dirección **api/admin-info/** con la vista **panel_admin_api**.

```
path('api/admin-info/', views.panel_admin_api, name='admin_api'),
```

Como las rutas de products ya están incluidas bajo **products/**, utilizaremos la siguiente dirección completa en Postman:

```
GET http://localhost:8000/products/api/admin-info/
```

El endpoint solamente consulta si el usuario autenticado tiene acceso administrativo, por eso utilizamos el método **GET**.



Probar con un usuario no administrador

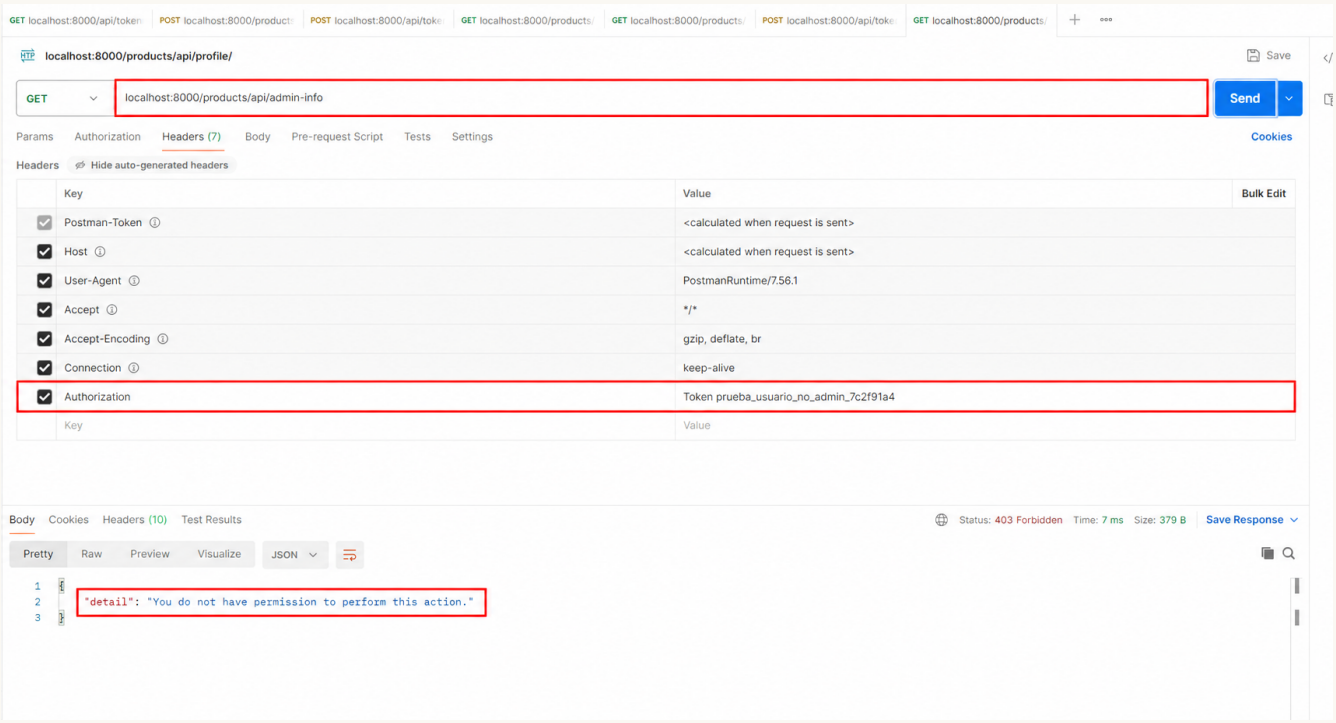
Paso 7 · Resultado esperado: 403 Forbidden

Levantamos el servidor con **py manage.py runserver**. En Postman realizamos un GET al nuevo endpoint y enviamos en Headers el token de un usuario registrado que no tenga activado **Staff status**.

```
Authorization: Token prueba_usuario_no_admin_7c2f91a4
```

TokenAuthentication reconoce al usuario, pero IsAdminUser determina que no posee el permiso requerido. DRF responde con **403 Forbidden** y genera automáticamente el mensaje **You do not have permission to perform this action.**

Este resultado confirma que el usuario está autenticado, pero no está autorizado para ingresar al endpoint administrativo.



Probar con un usuario administrador

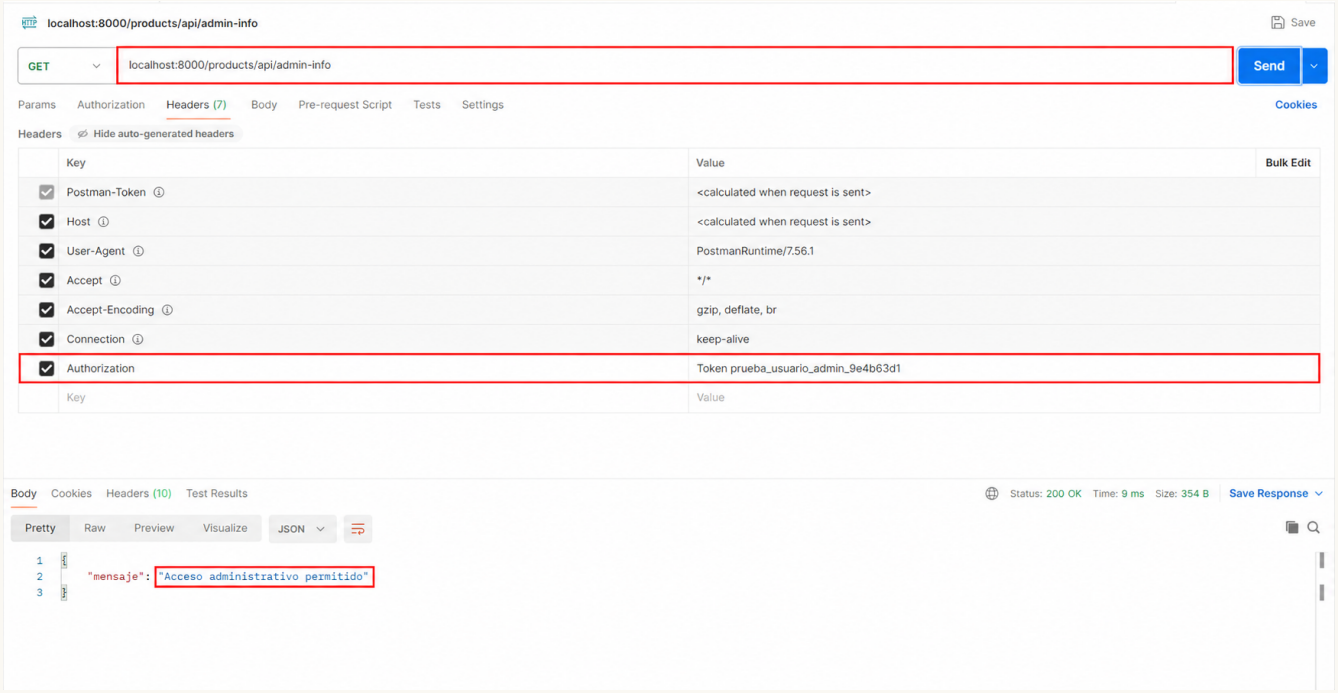
Paso 7 · Resultado esperado: 200 OK

Repetimos el GET con el token de un usuario que tenga activado **Staff status**. El header conserva el mismo formato; solamente cambia el valor del token.

```
Authorization: Token prueba_usuario_admin_9e4b63d1
```

Esta vez IsAdminUser permite que la función se ejecute. Postman muestra el estado **200 OK** y la respuesta contiene el mensaje **Acceso administrativo permitido**.

Con las dos pruebas comprobamos la diferencia entre autenticarse y tener permiso: ambos usuarios poseen un token válido, pero solamente el usuario administrador puede consultar esta ruta.



Crear un contador de sesión

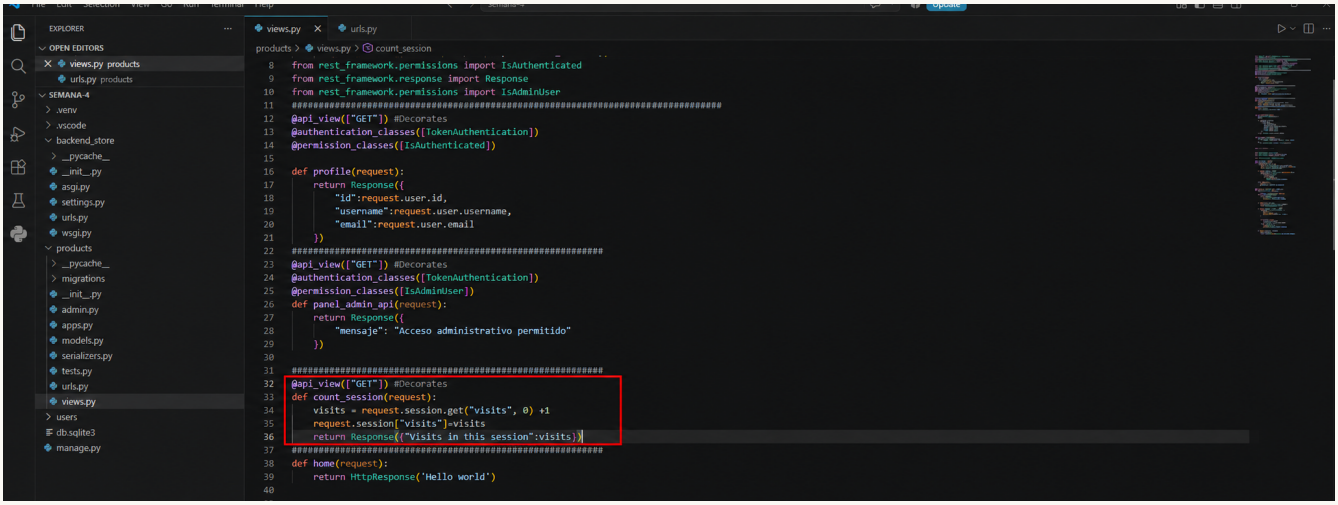
Paso 8 · Guardar y recuperar datos con request.session

Ahora creamos un endpoint que contará cuántas veces se ha consultado dentro de la misma sesión. Para esta demostración utilizamos **request.session**, donde Django puede conservar datos entre solicitudes relacionadas.

```
@api_view(['GET'])
def count_session(request):
visits = request.session.get('visits', 0) + 1
request.session['visits'] = visits
return Response({'Visits in this session': visits})
```

1. request.session.get('visits', 0) recupera el valor guardado. Si no existe todavía, utiliza 0.
2. Sumamos 1 y guardamos el resultado nuevamente en la sesión.
3. Response devuelve el contador actualizado en formato JSON.

No estamos contando inicios de sesión reales del usuario. Este ejemplo cuenta las consultas realizadas al endpoint mientras el navegador conserve la misma sesión.



Registrar y probar el contador

Paso 8 · Agregar la URL y recargar la página

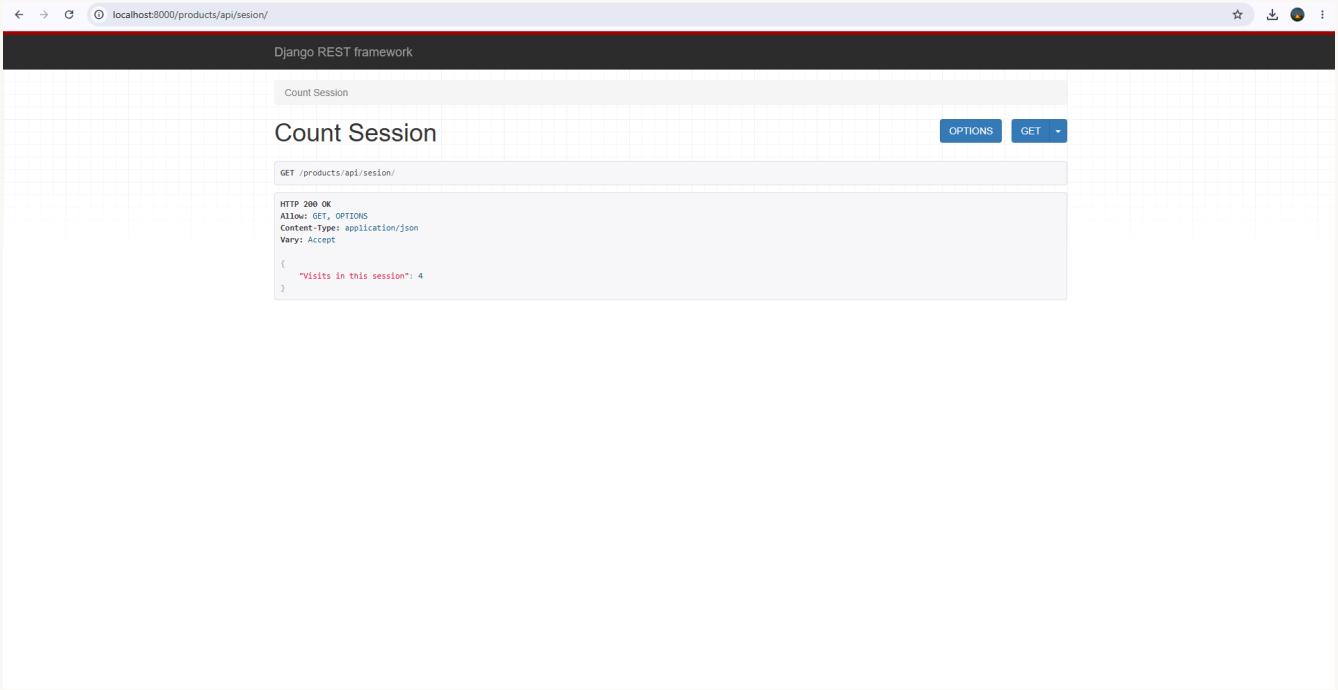
En **products/urls.py** agregamos la ruta para conectar el endpoint con la dirección **api/sesion/**.

```
path('api/sesion/', views.count_session, name='count_session'),
```

Después levantamos el servidor y abrimos la dirección en el navegador:

```
http://localhost:8000/products/api/sesion/
```

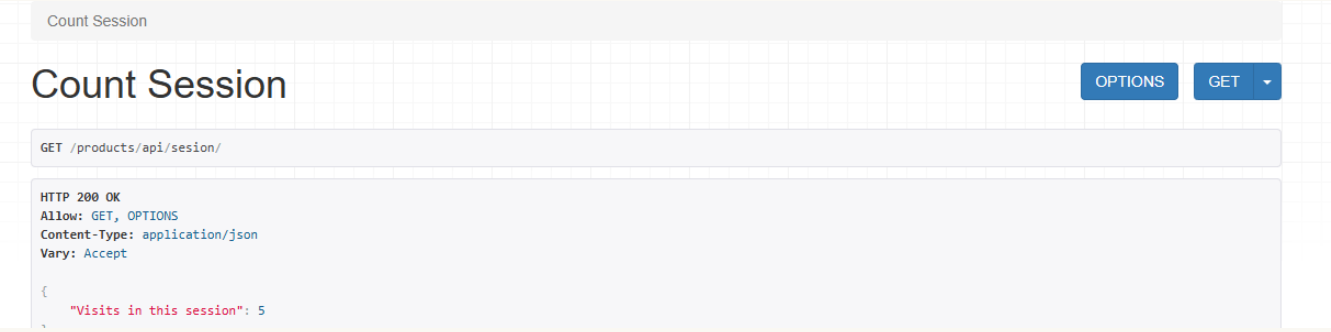
En la primera captura aparece el valor **4** porque la página ya se había recargado varias veces antes de tomarla. Cada nueva recarga realiza otra solicitud GET, recupera el valor anterior de la sesión y suma 1.



Comprobar que la sesión recuerda el valor

Paso 8 · El contador aumenta de 4 a 5

Al recargar nuevamente, el contador cambia de 4 a **5**, lo que confirma que Django recordó el dato entre ambas solicitudes.



El valor puede comenzar nuevamente si se elimina la cookie de sesión, se utiliza otro navegador o perfil, o la sesión expira.

Probar que el precio sea mayor que cero

Paso 9 · Crear una prueba para ProductSerializer

Nos dirigimos al archivo **products/tests.py** para comprobar que el serializer no acepte productos cuyo precio sea igual o menor que cero.

```
from django.test import TestCase
from .serializers import ProductSerializer

class ProductSerializerTest(TestCase):
def test_price_higher_zero(self):
data = {
'name': 'Mouse',
'description': 'Mouse usb',
'price': 0,
'stock': 5,
'asset': True,
}
serializer = ProductSerializer(data=data)
self.assertFalse(serializer.is_valid())
self.assertIn('price', serializer.errors)
```

La prueba envía **price: 0**. **assertFalse** confirma que los datos sean inválidos y **assertIn** comprueba que el serializer devuelva el error en el campo **price**.

